Lecture 15 Practice Questions and Solutions

Question 1 (q1_sol.py)

You have your first job as a software engineer! You work at a children's education software company, and you are in charge of checking if a word contains the letter *m*. The previous developer (who has been promoted to work with *n*) has left you with this function:

```
def has_m_print(word):
    for letter in word:
        if letter == 'm':
            print('this word has an "m"')

# test code
has_m_print('tumble')  # this word has an "m"
has_m_print('Map')     # nothing printed
has_m_print('apple')   # nothing printed
has_m_print('')        # nothing printed
```

Your task is to update the function as follows:

- make it find both lowercase `m` and uppercase `M`
- instead of printing a message, have it return `True` if the word contains an m (either lowercase or uppercase), and `False` otherwise; you should probably change the name of the function
- convert and update the test cases to work with this new function

Sample Solution

```
def has_m(word):
    for letter in word:
        if letter == 'm' or letter == 'M':
            return True
    return False

# test code
print(has_m('tumble'))  # True
print(has_m('Map'))     # True
print(has_m('apple'))   # False
print(has_m(''))        # False
```

Question 2 (q2_sol.py)

Rewrite the `is_prime` function so it uses a while loop instead of a for loop. Test it with the given testing code (you can keep the for-loop for the testing code --- or rewrite it to a while loop for extra practice).

```
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

#
# test code
#
n = 1000  # 25 primes less than 100, 168 primes less than 1000
num_primes = 0
for i in range(2, n):
    if is_prime(i):
        print(i, end=' ')
        num_primes += 1

print()
print()
print(f'Total number of primes less than {n}: {num_primes}')
```

Sample Solution

```
def is_prime(n):
    if n <= 1:
        return False
    i = 2
    while i < n:
        if n % i == 0:
            return False
        i += 1
    return True

#
# test code
#
n = 1000  # 25 primes less than 100, 168 primes less than 1000
num_primes = 0
for i in range(2, n):
    if is_prime(i):
        print(i, end=' ')
        num_primes += 1
print(f'\n\nTotal number of primes less than {n}: {num_primes}')
```

Question 3 (q3_sol.py)

q4_sol.py

During wildfire season, air quality is reported using the Air Quality Health Index (AQHI). Here's a simplified version of the advice for strenuous outdoor activity:

```
| AQHI      | At-risk people   | Everyone else    |
|-----------|------------------|------------------|
| below 1   | Invalid          | Invalid          |
| 1 to 3    | Low: normal      | Low: normal      |
| 4 to 6    | Moderate: reduce | Moderate: normal |
| 7 to 10   | High: avoid      | High: reduce     |
| above 10  | Very high: avoid | Very high: avoid |
```

Write a program that asks for the AQHI reading (as an int) and whether the user is at risk (y or n), and then prints the advice from the table.

(At-risk people include people with heart or lung problems, the elderly, and children.)

Sample Run 1

```
AQHI? 5
At risk (y/n)? n
Moderate: normal
```

Sample Run 2

```
AQHI? 8
At risk (y/n)? y
High: avoid
```

Sample Solution

```
aqhi = int(input('AQHI? '))
at_risk = input('At risk (y/n)? ')

if aqhi < 1:
    print('Invalid')
elif aqhi <= 3:
    print('Low: normal')
elif aqhi <= 6 and at_risk == 'y':
    print('Moderate: reduce')
elif aqhi <= 6:
    print('Moderate: normal')
elif aqhi <= 10 and at_risk == 'y':
    print('High: avoid')
elif aqhi <= 10:
    print('High: reduce')
```

```
else:
    print('Very high: avoid')
```